-- DEMO 7.1 – Отображения

```
-- Создаем отображение
CREATE VIEW SalesLT.vCustomerAddress
AS
SELECT C.CustomerID, FirstName, LastName, AddressLine1, City, StateProvince
FROM
SalesLT.Customer C JOIN SalesLT.CustomerAddress CA
ON C.CustomerID=CA.CustomerID
JOIN SalesLT.Address A
ON CA.AddressID=A.AddressID

-- Запрос к отображению
SELECT CustomerID, City
FROM SalesLT.vCustomerAddress

-- Соединение отображения и таблицы
SELECT c.StateProvince, c.City, ISNULL(SUM(s.TotalDue), 0.00) AS Revenue
FROM SalesLT.vCustomerAddress AS c
LEFT JOIN SalesLT.SalesOrderHeader AS s
ON s.CustomerID = c.CustomerID
GROUP BY c.StateProvince, c.City
ORDER BY c.StateProvince, Revenue DESC;
```

-- DEMO 7.2 – Временные таблицы и переменные

```
-- Временная таблица
CREATE TABLE #Colors
(Color varchar(15));

INSERT INTO #Colors
SELECT DISTINCT Color FROM SalesLT.Product;

SELECT * FROM #Colors;

-- Табличная переменная
DECLARE @Colors AS TABLE (Color varchar(15));

INSERT INTO @Colors
SELECT DISTINCT Color FROM SalesLT.Product;

SELECT * FROM @Colors;

-- Новый пакет
SELECT * FROM #Colors;

SELECT * FROM @Colors; -- сейчас вне области видимости
```

-- DEMO 7.3 – Табличные функции

```
CREATE FUNCTION SalesLT.udfCustomersByCity
(@City AS VARCHAR(20))
RETURNS TABLE
AS
RETURN
(SELECT C.CustomerID, FirstName, LastName, AddressLine1, City, StateProvince
 FROM SalesLT.Customer C
       JOIN SalesLT.CustomerAddress CA ON C.CustomerID=CA.CustomerID
       JOIN SalesLT.Address A ON CA.AddressID=A.AddressID
 WHERE City=@City);
```

```sql
SELECT * FROM SalesLT.udfCustomersByCity('Bellevue')
```

-- DEMO 7.4 – Производные таблицы

```sql
SELECT Category, COUNT(ProductID) AS Products
FROM
       (SELECT p.ProductID, p.Name AS Product, c.Name AS Category
        FROM SalesLT.Product AS p
        JOIN SalesLT.ProductCategory AS c
        ON p.ProductCategoryID = c.ProductCategoryID) AS ProdCats
GROUP BY Category
ORDER BY Category;
```

-- DEMO 7.5 – Обобщенные табличные выражения (CTE)

```sql
-- Использование CTE
WITH ProductsByCategory (ProductID, ProductName, Category)
AS
(
       SELECT p.ProductID, p.Name, c.Name AS Category
        FROM SalesLT.Product AS p
        JOIN SalesLT.ProductCategory AS c
        ON p.ProductCategoryID = c.ProductCategoryID
)

SELECT Category, COUNT(ProductID) AS Products
FROM ProductsByCategory
GROUP BY Category
ORDER BY Category;


-- Рекурсивное CTE
SELECT * FROM SalesLT.Employee

-- Используем CTE для выполнения рекурсии
WITH OrgReport (ManagerID, EmployeeID, EmployeeName, Level)
AS
(
       -- Корневой запрос
       SELECT e.ManagerID, e.EmployeeID, EmployeeName, 0
       FROM SalesLT.Employee AS e
       WHERE ManagerID IS NULL

       UNION ALL

       -- Рекурсивный запрос
       SELECT e.ManagerID, e.EmployeeID, e.EmployeeName, Level + 1
       FROM SalesLT.Employee AS e
       INNER JOIN OrgReport AS o ON e.ManagerID = o.EmployeeID
)

SELECT * FROM OrgReport
OPTION (MAXRECURSION 3);
```